Guía de estudio — Exposición final (Tu RAG, con cara)

Para estudiar, no para leer en voz alta: la idea es que lo digas con tus palabras.

1. Guion de la demo (en este orden)

1. **Mi corpus.** Financiación, garantía y service, políticas de venta y catálogo de modelos de PROSEBA automotores, una concesionaria de autos. Lo usaría un cliente que quiere saber cuánto necesita de anticipo, qué cubre la garantía o qué auto le conviene, sin buscar en varios documentos. Elegí tres documentos (en vez de uno o dos) para poder hacer preguntas que a veces mezclan temas y ver cómo responde la búsqueda en ese caso.
2. **El front.** El asesor virtual "Pancho" de PROSEBA automotores, con estilo oscuro tipo Ferrari (fondo casi negro, rojo solo como acento, esquinas rectas). Tiene un header grande con el auto que se afina a una sola línea al hacer scroll, el chat con el avatar de Pancho, un indicador "escribiendo…" mientras espera, y guarda el historial en pantalla. En cada respuesta hay un desplegable **"Ver los documentos consultados"** con los fragmentos que trajo la búsqueda: su similitud, una barra con la raya roja del umbral, si fue *usado* o *descartado por el umbral*, y el texto. En el lateral están las **"Preguntas para probar"** (se tocan y se envían solas) y "Cómo responde Pancho". Cuando no hay dato, sale un globo ámbar con el sello **"No figura en los documentos"** y la mejor similitud encontrada.
3. **Pregunta que funciona:** "*¿Cuánto necesito de anticipo para financiar un auto?*" → 30% mínimo, fuente `financiacion.md — Planes de financiación propia` (similitud 0,646; en la pantalla a veces "Anticipo y cuotas" queda apenas primero, 0,64, y "Planes de financiación propia" segundo: las dos están en los 3 resultados y el modelo cita la correcta). **Abrir el documento en esa sección y mostrar que dice lo mismo.**
4. **Pregunta que no está:** "*¿Quién ganó el último mundial?*" → similitud 0,247, muy debajo del umbral. En pantalla sale el globo ámbar "No figura en los documentos" con la nota "Ningún documento llegó al umbral (0.5)…", **sin llamar al modelo.** Abrir "Ver los documentos consultados" y mostrar la similitud contra la raya del umbral.
5. **Pregunta que falla (en el límite):** "*¿Cuánto cuesta el service de la Vela 1.6?*"
6. La **búsqueda falló a medias**: trajo el modelo Vela 1.6 (0,70), la de mantenimiento programado (0,525) y la de reserva (0,523), todas por encima del umbral, pero ninguna tiene el precio puntual.
7. Lo salvó el **modelo**: el prompt le exige usar solo los fragmentos y decir "No lo encuentro" si no está, y en general lo hace, en vez de inventar un número. Ojo: el modelo no es 100% determinista; a veces, en vez de "No lo encuentro", contesta solo lo que sí dice el documento (el primer service es gratis y el costo depende del modelo) **sin inventar ningún precio**. Las dos respuestas son aceptables: lo importante es que nunca da un número inventado.

8. Por qué pasa: el documento de garantía y service dice textualmente que el costo "depende del modelo y se informa al momento del turno"; no hay un precio fijo que citar, y el embedding igual acerca la pregunta al modelo correcto porque el tema general coincide.
9. **Extra — pregunta que cruza dos documentos y funciona:** "*Compré un usado certificado y lo quiero entregar como parte de pago de un 0km: ¿qué garantía tiene el usado certificado y hasta qué antigüedad me lo reciben?*" → la búsqueda trae un fragmento de `garantia-service.md` (garantía de usados: 6 meses o 10.000 km) y uno de `financiacion.md` (no se reciben usados de más de 15 años), y el modelo combina los dos en una sola respuesta, citando las dos fuentes. Es el extra de "más de un documento, con una pregunta que necesita datos de los dos".
10. **Mis estrategias** (ver sección 2).

2. Mis estrategias, con evidencia

- **Troceado: por título Markdown** (`marca.mjs` : `/^#{1,3} .+/`). Cada sección del documento es un fragmento, y esa sección es la fuente que se cita. En el catálogo, cada modelo de auto es un `###` propio, así que queda como su propio fragmento citable. Salieron 26 fragmentos (7 + 7 + 12), todos con sección reconocida (el más largo, 729 caracteres). Si un fragmento pasara de 2000, se corta entre párrafos y repite el título; lo que se pierde es el hilo entre pedazos.
- **Umbral: 0,50.** Medí 16 preguntas (11 que están y 5 que no). Las que están dieron entre 0,505 y 0,786; las que no, entre 0,247 y 0,693. Se pisan: la peor que está (caballos de fuerza de la Travesía, 0,505) queda por debajo de la mejor que no está (service de la Vela, 0,693).
- **Qué error elegí cometer:** dejar pasar una pregunta que no está (la del service de la Vela) antes que descartar una respuesta válida. Un falso "no" le niega información real al cliente y no tiene arreglo; un falso "sí" todavía lo puede frenar el modelo con el prompt.
- **Prompt (`prompt.txt`):** lo adapté a la concesionaria y le di personaje (Pancho, asesor de ventas, voseo). Mantuve las tres restricciones: usar *exclusivamente* los fragmentos, la frase exacta cuando no está, y el renglón `Fuente:` con archivo y sección.
- **Lo que medí:** la sección correcta salió primera en 11 de 11 preguntas que están.
- **Lo que probé y descarté / cambié:** al cambiar el corpus del club por el de la concesionaria tuve que recalibrar el umbral desde cero: el 0,58 que había medido para el club no servía (habría descartado la pregunta de la Travesía, que sí está). El modelo de chat (`gemini-3.5-flash`) y el de embeddings (`bge-m3`) no cambiaron.

3. Preguntas sobre mi sistema

¿Qué muestra el front y de dónde saca los datos? El navegador solo manda la pregunta a `POST /preguntar` . El servidor (`servidor.mjs`) devuelve `respuesta` , `llamoAlModelo` , `umbral` y la lista de `fragmentos` (etiqueta, similitud, si fue útil y el texto). El front dibuja con eso las fuentes, la barra de similitud y la raya del umbral. `GET /estado` dice si la base y Ollama están vivos (el punto "En línea"). No hay lógica de RAG en el front: todo se decide en el servidor.

¿Qué corre en mi compu y qué en la nube? ¿Por qué? En mi compu: Postgres con pgvector (en Docker), el modelo de embeddings bge-m3 (en Ollama) y el servidor Node. En la nube: solo el modelo que escribe la respuesta (Gemini de Google). Un modelo de embeddings lee el texto una sola vez y corre en cualquier compu; un modelo de chat genera token por token y necesita mucho más hardware. Además indexar hace muchos pedidos (uno por lote) y así no gasto el cupo gratuito de Google.

¿Qué pasa si cambio el modelo de embeddings y no vuelvo a indexar? Si la dimensión es distinta, la base rechaza el dato con error (`expected 1024 dimensions…`). Si la dimensión coincide, no hay error pero los resultados no significan nada: cada modelo ubica los textos en un mapa distinto, y la pregunta y el corpus tienen que estar en el mismo. Además habría que volver a calibrar el umbral.

¿Qué pasa si corro `indexar.mjs` dos veces? ¿Cómo lo resolví? Un `INSERT` agrega, no reemplaza: quedarían 52 filas con cada fragmento duplicado. Lo resolví con `TRUNCATE TABLE fragmentos RESTART IDENTITY` al principio de `indexar.mjs`. Sigue dando 26 sin importar cuántas veces lo corra. (Los sistemas grandes hacen actualización incremental.)

¿Por qué la clave de API no puede ir en el navegador? ¿Dónde está en mi front? Cualquiera la lee con F12 (código, pestaña de red). En mi front el navegador solo manda la pregunta a `/preguntar`; la clave está en `.env`, la lee `api.mjs` y corre en el servidor (`servidor.mjs`), nunca viaja a la página.

¿Por qué el `ORDER BY` va por la distancia y no por la similitud? Da el mismo orden, pero el índice HNSW solo se usa cuando el `ORDER BY` es exactamente la distancia (`embedding <=> $1`). La similitud (`1 - distancia`) la calculo solo para mostrarla y comparar con el umbral.

¿Qué error preferí cometer con mi umbral, y por qué? Dejar pasar una pregunta que no está, antes que descartar una que sí. Ver sección 2.

¿Cómo cambiaste el sistema de tema (del club a la concesionaria)? ¿Qué tocaste y qué no? Reemplacé el corpus en `corpus/`, adapté el primer párrafo de `prompt.txt` (persona y tema, sin tocar las reglas), reescribí `pruebas.json` con preguntas nuevas, reindexé y volví a calibrar el umbral con `distribucion.mjs`. No toqué el núcleo: `api.mjs`, `buscar.mjs`, `rag.mjs`, `db.mjs`, `trocear.mjs` y la marca (`/^#{1,3} .+/`) siguen siendo los mismos, porque mi corpus nuevo también usa títulos de Markdown.

Mostrame en el código dónde se arma el prompt. `armarPrompt()` en `rag.mjs`. Toma la plantilla de `prompt.txt` y reemplaza `{{fragmentos}}` y `{{pregunta}}`. Cada fragmento va con su encabezado `archivo — sección`, que es lo que el modelo cita en `Fuente:`.

4. Preguntas sobre cómo funciona un RAG

¿Qué es un embedding? ¿Por qué textos sin palabras en común pueden quedar cerca? Una lista de números (1024 en bge-m3) que representa el significado del texto. El modelo se entrenó para que textos con significado parecido den vectores que apuntan para el mismo lado, aunque usen palabras distintas

("cuánto necesito para empezar a pagar" y "anticipo mínimo"). Por eso sirve donde la búsqueda por palabras de la clase 5 fallaba.

¿Qué mide la similitud coseno? El ángulo entre dos vectores, sin importar su largo: 1 si apuntan igual, 0 si son perpendiculares. En pgvector se usa la distancia coseno (`<=>`), que es 1 menos la similitud.

¿Por qué no alcanza con pegarle el corpus entero al modelo en cada pregunta? Cuesta más (se paga por cada token), es más lento, tiene un límite de contexto que un corpus grande excede, y el modelo se distrae con información irrelevante. Con búsqueda le paso solo los 3 fragmentos que importan, y además sé de dónde salió la respuesta.

¿Cómo encuentra HNSW los más cercanos sin mirarlos a todos? ¿Por qué es aproximado? Arma un grafo donde cada vector se conecta con unos vecinos cercanos, en varios pisos (arriba pocos puntos y conexiones largas, abajo todos y conexiones cortas). La búsqueda arranca arriba, salta de vecino en vecino al que está más cerca de la pregunta y baja de piso para afinar. Puede quedar trabada en un punto cercano que no es el más cercano de todos: cambia un poco de exactitud por mucha velocidad. Con 26 fragmentos no se nota; importa con miles o millones.

¿Quién evita que el modelo invente: la búsqueda, el umbral o el prompt? Los tres, en capas. El **umbral** corta antes de llamar al modelo si nada se parece (el mundial). La **búsqueda** limita lo que el modelo puede ver. El **prompt** le ordena usar solo esos fragmentos y decir "No lo encuentro" si no está (lo que salvó la pregunta del service de la Vela). Ninguna sola es perfecta, y esa pregunta lo demuestra: la búsqueda dejó pasar fragmentos sin la respuesta exacta, y fue el prompt el que evitó que el modelo inventara un precio.

¿Qué le falta a mi sistema para usarse en serio? (las cuatro cosas de la clase 8) 1. **Reranking:** un modelo más preciso que reordene los fragmentos mirando pregunta y fragmento juntos (habría distinguido mejor "precio del auto" de "precio del service"). 2. **Búsqueda híbrida:** combinar significado con palabras exactas (BM25), útil para nombres de modelos y datos puntuales. 3. **Actualización incremental:** reindexar solo los documentos que cambiaron, no todo. 4. **Medición formal:** un conjunto grande de preguntas con su respuesta correcta para medir el porcentaje de aciertos (yo medí 16).

5. Antes de que te toque (checklist)

- [ ] Docker Desktop abierto y `docker start rag-db`
- [ ] Ollama abierto
- [ ] `.env` con la clave de Google y conexión a internet
- [ ] `node --env-file=.env servidor.mjs` y http://localhost:3000 probado esa mañana
- [ ] Mis tres preguntas anotadas (están en "Preguntas para probar" del lateral): anticipo para financiar / mundial / service de la Vela 1.6
- [ ] `financiacion.md` abierto en "Planes de financiación propia" para mostrar la fuente
- [ ] En el chat, abrir "Ver los documentos consultados" para mostrar similitud y umbral
- [ ] Plan B: capturas o video corto
- [ ] Fichas 1 a 5 completas a mano

Si Google da error 503 ("alta demanda") o 429 (cupo agotado), reintentar con el botón "Reintentar" del chat. Si persiste, cambiar `MODELO_CHAT` en `.env` por otro modelo disponible (por ejemplo `gemini-flash-lite-latest`) y reiniciar el servidor. El modelo de chat puede variar entre el original (`gemini-3.5-flash`) y el que esté en uso en `.env`: las respuestas cambian un poco de redacción, pero los fragmentos y las similitudes son los mismos.

Fichas de la clase 8 — completadas con los datos reales de mi sistema

Corpus: Financiación + Garantía y Service + Políticas de Venta y Catálogo de PROSEBA automotores (concesionaria de autos). Los números son los que salieron de correr mi propio sistema. Pasalas a mano a las fichas y reescribí las explicaciones con tus palabras.

Ficha 1 — La dimensión (paso 3)

Antes de correr prueba.mjs — ¿Cuántos números va a devolver bge-m3? ¿Por qué? Yo calculaba que no iban a ser los mismos 1536 de Google: cada modelo de embeddings define su propia cantidad de números, y bge-m3 es otro modelo. Supuse que serían menos de 1536 (alrededor de 1000).

Después de correr — ¿Cuántos devolvió? ¿En qué dos lugares tengo que escribir ese número? Devolvió **1024** números. Hay que escribirlo en dos lugares: en el `.env` (`DIMENSIONES=1024`) y en `esquema.sql` (`VECTOR(1024)`). El `.env` controla lo que sale del modelo y la columna controla lo que entra a la base: tienen que coincidir siempre. (Este número no cambió al cambiar de tema porque el modelo de embeddings, bge-m3, es el mismo; lo que cambió es el corpus.)

Ficha 2 — Mi estructura (paso 5)

Pregunta	Mi respuesta
La marca que escribí	`/^#{1,3} .+/` (los títulos de Markdown: #, ## y ###)
Fragmentos que salieron	**26** (7 de financiación + 7 de garantía/service + 12 de políticas y catálogo)
Fragmentos con sección reconocida	**26 de 26**
El más corto y el más largo (caracteres)	**149** y **729**

¿Las secciones son las que esperaba? Si alguna parte quedó sin reconocer, ¿qué tiene de distinto su comienzo? Sí, todas son las esperadas ("Planes de financiación propia", "Garantía de fábrica (0km)", "Reserva y seña", y cada modelo del catálogo como su propia sección: "Sedán Vela 1.6", "Pick-up Cordillera 4x4 Diésel", etc.). No quedó ninguna parte sin reconocer, porque todo mi corpus está escrito con títulos de Markdown. El catálogo usa `###` para cada modelo, así que cada auto queda como un fragmento propio con su propia sección.

Si el más largo pasa de 2000 caracteres, ¿qué le hace la función? ¿Qué se pierde? El más largo tiene 729, así que no pasa. Si pasara, la función lo corta entre párrafos en pedazos de hasta 2000 caracteres y repite el título de la sección en cada pedazo. Lo que se pierde es el hilo: el segundo pedazo ya no "ve" lo que dijo el primero.

Si mi marca es null: ¿por qué mi corpus no tiene una marca que sirva? ¿Qué problema de la clase 7 vuelve a aparecer? No aplica: mi marca no es null. Si lo fuera, el corpus se cortaría por tamaño fijo y los fragmentos quedarían sin sección, así que la fuente diría "fragmento 12" en vez de un nombre. Además, un corte por tamaño parte las ideas por la mitad, el problema de la clase 7.

Ficha 3 — Indexar otra vez (paso 7)

Antes de correr — Filas después del paso 6: **26**. Si corro `indexar.mjs` otra vez sin cambiar nada, ¿cuántas filas va a haber? ¿Por qué? Si no tuviera el `TRUNCATE`, habría **52**: un `INSERT` agrega filas, no reemplaza, así que cada fragmento quedaría dos veces.

Después de correr — Mi `indexar.mjs` ya tiene `TRUNCATE TABLE fragmentos RESTART IDENTITY` al principio (lo agregué la primera vez que armé este sistema, con el tema del club, y se mantuvo sin cambios al cambiar de tema). Por eso, aunque lo corrí varias veces al reindexar el corpus nuevo, siempre volvió a dar **26** filas, nunca más. Sin ese `TRUNCATE`, entre los tres primeros resultados de una pregunta aparecería el mismo fragmento dos veces (con la misma similitud), y el modelo recibiría el mismo texto repetido con un fragmento útil menos. En las clases 6 y 7 esto no pasaba porque `writeFile` reemplaza el archivo entero: indexar veinte veces dejaba siempre un solo índice. La base, en cambio, acumula todo lo que se le manda, así que soy responsable de no repetir.

Ficha 4 — Dónde cortar (paso 9) — se conserva

Mi UMBRAL anterior (tema del club): 0,58. **¿Me sirve para hoy? ¿Por qué?** No. El umbral depende del corpus y de cómo el embedding ubica sus textos: un número medido con el reglamento del club no tiene por qué servir con los documentos de la concesionaria, aunque el modelo de embeddings sea el mismo. Hay que volver a medir cada vez que cambia el corpus.

Similitud del mejor fragmento para cada pregunta:

Pregunta	¿Está?	Similitud
¿cuánto tarda la entrega de un 0km que está en stock?	sí	0,786
¿cuánto tengo que pagar de seña para reservar un auto?	sí	0,765
¿cuántos años de garantía tiene un 0km?	sí	0,762
Compré un usado certificado y lo quiero entregar como parte de pago de un 0km: ¿qué garantía tiene y hasta qué antigüedad me lo reciben? (cruza dos documentos)	sí	0,754
¿puedo entregar mi auto usado como parte de pago?	sí	0,749
¿qué consumo de combustible tiene la pick-up Cordillera?	sí	0,724
¿cuánto necesito de anticipo para financiar un auto?	sí	0,646
¿qué documentación tengo que llevar para comprar un auto?	sí	0,643
¿cada cuánto hay que hacer el service?	sí	0,614

Pregunta	¿Está?	Similitud
¿me dan auto de cortesía mientras hacen el service?	sí	0,575
¿cuántos caballos de fuerza tiene la Travesía?	sí	0,505
¿cuánto cuesta el service de la Vela 1.6?	no	0,693
¿hacen envíos a domicilio del auto comprado?	no	0,586
¿tienen autos eléctricos en el catálogo?	no	0,489
¿cómo se hace una milanesa?	no	0,374
¿quién ganó el último mundial?	no	0,247

La recta (X = está, O = no está, ^ = mi umbral 0,50):

```
                              X X   XX  X XX X  X
             0     0   0      0
  ------------------------------------------------------
                           ^
  0,0  0,1  0,2  0,3  0,4  0,5  0,6  0,7  0,8  0,9  1
```

Dibujala a mano en la ficha: cruces para las que están, círculos para las que no, y una flecha en 0,50.

1. **Con el umbral anterior (0,58, calibrado para el club):** si lo hubiera usado hoy, descartaría la pregunta de los caballos de fuerza de la Travesía (0,505, por debajo de 0,58) y dejaría pasar igual la del service de la Vela (0,693) y la de envíos a domicilio (0,586). Conclusión: un umbral pensado para otro corpus puede dejar pasar basura y además descartar respuestas buenas, y por eso hay que volver a medir.
2. **¿Hay un hueco limpio?** No, se pisan: la cruz más baja (0,505, caballos de la Travesía) está por debajo del círculo más alto (0,693, service de la Vela 1.6).
3. **Mi umbral nuevo: 0,50.**
4. **Las preguntas del lado equivocado:** "service de la Vela 1.6" (0,693) y "envíos a domicilio" (0,586) pasan el umbral aunque no están. Les pasa porque el embedding capta el tema general (el auto, la entrega) y no el dato puntual que falta (el precio del service, si hay envío). Elegí 0,50 para no perder ninguna respuesta válida: prefiero dejar pasar una pregunta que no está (que el prompt todavía puede frenar) antes que descartar una que sí está.

Ficha 5 — Mis cuatro preguntas (paso 10) — se conserva para la exposición

#	Pregunta	Similitud del primero	Fuente que citó	¿Correcta?	Si falló: ¿búsqueda o modelo?
1	¿cuánto necesito de	0,646	financiacion.md — Planes de	Sí (30% mínimo)	—

#	Pregunta	Similitud del primero	Fuente que citó	¿Correcta?	Si falló: ¿búsqueda o modelo?
	anticipo para financiar un auto?		financiación propia		
2	¿cuántos años de garantía tiene un 0km?	0,762	garantia-service.md — Garantía de fábrica (0km)	Sí (3 años o 100.000 km)	—
3	¿quién ganó el último mundial?	0,247	ninguna (no se llamó al modelo)	Sí: "No lo encuentro en los documentos."	— (el umbral la cortó)
4	¿cuánto cuesta el service de la Vela 1.6?	0,693	trajo "Sedán Vela 1.6", "Reserva y seña" y "Service y mantenimiento"	Contestó bien sin inventar precio ("No lo encuentro" o solo lo que dice el documento: el primer service es gratis y el costo depende del modelo; varía entre corridas), pero por el motivo equivocado	Falló la **búsqueda** (el precio del service no está en ninguno de los tres fragmentos: el documento solo dice que depende del modelo); lo salvó el **modelo**

Para la exposición: - **Pregunta que funciona:** la del anticipo para financiar. Abrir `financiacion.md` en "Planes de financiación propia" y mostrar que dice 30% mínimo. - **Pregunta que no está y el sistema dice que no:** la del mundial (0,247, muy debajo del umbral, sin llamar al modelo). - **Pregunta que falla:** la del service de la Vela 1.6. La búsqueda trajo fragmentos relacionados (el modelo, la reserva, el plan de mantenimiento) pero ninguno tiene el precio puntual, porque el documento aclara que el costo del service "depende del modelo y se informa al momento del turno". El umbral los dejó pasar (0,693, 0,522 y 0,515, todos sobre 0,50), y el prompt, que obliga al modelo a usar solo los fragmentos, impidió que inventara un precio. Para arreglarlo de verdad haría falta reranking o búsqueda híbrida. - **Bonus — pregunta compuesta que también falla por búsqueda:** "si entrego mi usado como parte de pago y el 0km tiene garantía, ¿en qué casos se anula la garantía por no hacer el service?" mezcla dos temas en una sola oración; la búsqueda trae fragmentos de "Devoluciones y cambios", el título general de garantía y "Entrega de tu usado…", mientras que la respuesta real está en "Service y mantenimiento programado". El sistema dice correctamente que no lo encuentra, pero la causa es, otra vez, una falla de búsqueda y no que la respuesta no exista en el corpus.

Flujo completo del RAG — PROSEBA automotores

> Este documento explica cómo funciona el sistema de principio a fin, con definiciones de cada concepto técnico que aparece en el camino.

¿Qué es un RAG?

RAG = *Retrieval-Augmented Generation* (Generación con Recuperación de Información).

Es una técnica que combina dos cosas: 1. **Buscar** fragmentos relevantes en una base de datos propia. 2. **Generar** una respuesta usando un modelo de lenguaje (IA), pero *solo* basándose en lo que encontró.

La ventaja es que el modelo no inventa: solo puede responder con lo que está en tus documentos.

Vista general del flujo

```
Documentos (.md/.txt)
        ↓
   [Troceado]           ← Se parten en fragmentos pequeños
        ↓
   [Embedding]          ← Cada fragmento se convierte en un vector numérico
        ↓
   [Base de datos]      ← Los vectores se guardan en Postgres con pgvector
        ↓ ↓ ↓  (esto es el ÍNDICE — se hace una sola vez)

   Usuario pregunta
        ↓
   [Embedding]          ← La pregunta también se convierte en vector
        ↓
   [Búsqueda vectorial] ← Se buscan los fragmentos más parecidos al vector de la pregunta
        ↓
   [Umbral]             ← Si la similitud es muy baja, se descarta (evita inventar)
        ↓
   [Prompt]             ← Se arma un mensaje combinando los fragmentos + la pregunta
        ↓
   [Gemini (Google)]    ← El modelo genera la respuesta final
        ↓
   Respuesta al usuario
```

Fase 1: Indexación (se hace una sola vez)

Este es el proceso de preparar los documentos para que puedan buscarse.

Paso 1 — Los documentos del corpus

Los archivos `.md` dentro de la carpeta `corpus/` son la **fuente de conocimiento** del sistema: - `financiacion.md` - `garantia-service.md` - `politicas-y-catalogo.md`

Son documentos de texto que contienen toda la información que Pancho (el asistente) puede usar para responder.

Paso 2 — Troceado (`trocear.mjs`)

Los documentos son largos. No se pueden mandar completos al modelo en cada consulta, entonces se parten en **fragmentos** más pequeños.

¿Cómo se trocean? - **Por sección** (método principal): cada vez que aparece un título Markdown (`## Título`), empieza un fragmento nuevo. Si la sección es muy larga (más de 2000 caracteres), se corta en partes más chicas repitiendo el título para que el fragmento no quede huérfano de contexto. - **Por tamaño fijo** (plan B): si el corpus no tiene títulos, se cortan bloques de 800 caracteres con un solapamiento de 150 caracteres entre uno y el siguiente.

Cada fragmento tiene: | Campo | Descripción | |---|---| | `fuente` | De qué archivo viene (ej: `financiacion.md`) | | `seccion` | El título de la sección (ej: `Planes de financiación propia`) | | `posicion` | Número de orden dentro del archivo | | `texto` | El contenido del fragmento |

Paso 3 — Embedding (`api.mjs` → Ollama)

Acá ocurre la "magia". Cada fragmento de texto se convierte en un **vector de números**.

¿Por qué? Porque las computadoras no entienden texto directamente. Al convertir el texto en números, se puede comparar matemáticamente qué tan parecidos son dos fragmentos.

El modelo que hace esto se llama `bge-m3` y corre localmente en tu máquina con **Ollama**. Genera **1024 números** (dimensiones) por cada fragmento.

```
"PROSEBA automotores financia directamente hasta el 70%..."
         ↓  (bge-m3 en Ollama)
[ 0.023, -0.187, 0.441, 0.009, ... ]  ← 1024 números
```

Dos textos con significado similar van a tener vectores parecidos, aunque usen palabras distintas.

Paso 4 — Guardado en la base de datos (`db.mjs` → Postgres + pgvector)

Los fragmentos (texto + vector) se guardan en una tabla llamada `fragmentos` en **PostgreSQL**.

La extensión **pgvector** le agrega a Postgres la capacidad de guardar y buscar vectores eficientemente.

También se crea un **índice HNSW** sobre los vectores, que es una estructura que acelera enormemente las búsquedas (como el índice de un libro, pero para vectores).

```
-- Así se ve una fila en la tabla:
id | fuente              | seccion     | posicion | texto           | embedding
---+---------------------+-------------+----------+-----------------+-----------
 1 | financiacion.md     | Planes de financiación propia |    0     | "PROSEBA automotores
financia..." | [0.023, -0.187, ...]
```

Fase 2: Consulta (ocurre cada vez que el usuario pregunta)

Paso 5 — Embedding de la pregunta

Cuando el usuario escribe una pregunta, **se le aplica el mismo proceso de embedding** que a los fragmentos.

```
"¿Cuánto necesito de anticipo para financiar un auto?"
         ↓  (bge-m3 en Ollama)
[ 0.019, -0.201, 0.438, ... ]  ← 1024 números
```

Ahora tenemos la pregunta como vector y podemos compararla con los vectores de los fragmentos.

Paso 6 — Búsqueda vectorial (`buscar.mjs`)

Postgres compara el vector de la pregunta contra todos los vectores de la tabla `fragmentos` y devuelve los más cercanos (por defecto, los **3 más parecidos**, `k = 3`).

La comparación usa la **distancia coseno** (`<=>` en pgvector): mide el ángulo entre dos vectores. A menor ángulo → mayor similitud.

```
SELECT fuente, seccion, texto,
       1 - (embedding <=> $1::vector) AS similitud
FROM fragmentos
ORDER BY embedding <=> $1::vector
LIMIT 3
```

El resultado es una lista de fragmentos con su **puntaje de similitud** (un número entre 0 y 1, donde 1 es idéntico).

Paso 7 — Filtro por umbral (`rag.mjs`)

No todos los fragmentos encontrados son útiles. Si el más parecido tiene una similitud de 0.3, probablemente no tiene nada que ver con la pregunta.

El **UMBRAL** es un número mínimo de similitud (configurado en el `.env`). Si ningún fragmento supera el umbral, el sistema responde directamente: `"No lo encuentro en los documentos."` y **no llama al modelo de IA**.

Esto evita que el modelo invente respuestas cuando no hay información relevante.

Paso 8 — Armado del prompt (`rag.mjs` → `prompt.txt`)

Con los fragmentos útiles, se arma el **prompt**: el mensaje completo que se le manda al modelo de IA.

La plantilla (`prompt.txt`) define el personaje (Pancho) y las instrucciones:

```
Sos Pancho, asesor de ventas de PROSEBA automotores...
Respondé usando EXCLUSIVAMENTE los fragmentos que aparecen más abajo.
Si los fragmentos no contienen la respuesta, respondé exactamente: "No lo encuentro en los
documentos."

FRAGMENTOS:
--- financiacion.md — Planes de financiación propia ---
PROSEBA automotores financia directamente hasta el 70% del valor del vehículo...

PREGUNTA:
¿Cuánto necesito de anticipo para financiar un auto?
```

Paso 9 — Respuesta de Gemini (`api.mjs` → Google)

El prompt completo se manda a **Gemini** (modelo de Google) vía API. Gemini genera la respuesta final, respetando las instrucciones del prompt: personaje, tono, límite de dos oraciones, y citar la fuente.

```
Pancho responde:
"Necesitás un anticipo mínimo del 30% para acceder al plan propio.
Fuente: financiacion.md — Planes de financiación propia"
```

Paso 10 — Lo que ve el usuario (`public/index.html`)

El navegador solo manda la pregunta a `POST /preguntar`. El servidor devuelve la respuesta, si llamó o no al modelo, el umbral y la lista de fragmentos con su similitud. El front muestra:

- La respuesta de Pancho, con la hora y los chips de **fuente** (`Fuente: archivo — sección`).
- Si ningún fragmento llegó al umbral (o el modelo no encontró el dato): un globo ámbar con el sello **"No figura en los documentos"** y una nota con la mejor similitud (ej.: "Ningún documento llegó al umbral (0.5); el más parecido llegó a 0.486").
- Un desplegable **"Ver los documentos consultados"**: para cada fragmento, si fue *usado* o *descartado por el umbral*, su similitud, una barra con la raya roja del umbral y el texto completo. Es lo que se usa en la exposición para mostrar cómo decide el sistema.
- En el lateral, **"Preguntas para probar"** (se tocan y se envían) y **"Cómo responde Pancho"**.

La clave de Google nunca llega al navegador: queda en `.env` y la usa el servidor.

Mapa de archivos del proyecto

Archivo	Qué hace
`corpus/`	Los documentos fuente de conocimiento
`trocear.mjs`	Parte los documentos en fragmentos
`marca.mjs`	Define qué patrón de texto marca el inicio de un fragmento (títulos `#`, `##` y `###`)
`api.mjs`	Llama a Ollama (embeddings) y a Google (respuesta)
`db.mjs`	Conexión a Postgres
`crear-tabla.mjs`	Crea la tabla `fragmentos` y el índice HNSW en la base
`indexar.mjs`	Lee el corpus, trocéa, calcula embeddings y guarda todo en la base
`buscar.mjs`	Hace la búsqueda vectorial en SQL
`rag.mjs`	Orquesta todo: embedding → búsqueda → umbral → prompt → respuesta
`prompt.txt`	Plantilla del prompt con el personaje Pancho
`servidor.mjs`	Servidor web: sirve el chat en http://localhost:3000 y expone `/estado` y `/preguntar`
`public/index.html`	El front: una sola página (HTML, CSS y JS) con el chat de Pancho
`preguntar.mjs`	Versión de consola para hacer preguntas sin el front
`distribucion.mjs`	Ayuda a elegir el umbral correcto

Glosario de términos técnicos

Término	Definición simple
RAG	Técnica que busca información en documentos propios antes de responder con IA, para no inventar.
Embedding	Proceso de convertir texto en una lista de números que representa su significado semántico.
Vector	La lista de números que resulta del embedding. Texto similar → vectores cercanos.
Dimensiones	Cantidad de números en cada vector. En este proyecto: 1024.

Término	Definición simple
bge-m3	El modelo de embeddings que corre en Ollama. Convierte texto en vectores.
Ollama	Programa que corre modelos de IA localmente en tu computadora, sin internet.
Postgres / PostgreSQL	Base de datos relacional donde se guardan los fragmentos y sus vectores.
pgvector	Extensión de Postgres que le agrega soporte para guardar y buscar vectores.
Índice HNSW	Estructura de datos que acelera la búsqueda de vectores similares (como el índice de un libro).
Distancia coseno	Medida matemática del ángulo entre dos vectores. Menor ángulo = más similares.
Similitud	Número entre 0 y 1 que indica qué tan parecido es un fragmento a la pregunta. 1 = idéntico.
Umbral	Similitud mínima aceptada. Fragmentos por debajo de este valor se ignoran.
Corpus	El conjunto de documentos que forman la base de conocimiento del sistema.
Troceado / Chunking	Partir documentos largos en partes más pequeñas para poder indexarlas y buscarlas.
Fragmento / Chunk	Cada una de las partes en que se divide un documento.
Solapamiento	Zona de texto que se repite entre dos fragmentos consecutivos, para no perder contexto en los bordes.
Prompt	El mensaje completo que se le manda al modelo de IA, incluyendo contexto, instrucciones y pregunta.
Gemini	El modelo de lenguaje de Google que genera las respuestas finales del asistente.
LLM	*Large Language Model* — modelo de lenguaje grande, como Gemini. Genera texto a partir de un prompt.
Temperature 0	Configuración del modelo que hace las respuestas lo más consistentes y predecibles posible (sin creatividad).
API Key	Clave secreta que identifica al usuario ante el servicio de Google para poder usar Gemini.
Docker	Programa que corre aplicaciones en contenedores aislados. Acá se usa para correr Postgres.
`.env`	Archivo de configuración con variables secretas (claves, URLs, umbrales) que no se suben a git.

Anexo — Preguntas para hacerle a Pancho

Las 16 preguntas de `pruebas.json`, con la similitud medida del mejor fragmento y lo que se espera ver. Umbral: **0,50**. Las marcadas con ★ también están como botón en el lateral del chat ("Preguntas para probar").

A. Funcionan: la respuesta está en los documentos

Pregunta	Sección que debería citar	Similitud
¿Cuánto tarda la entrega de un 0km que está en stock?	Entrega del vehículo	0,786
★ ¿Cuánto tengo que pagar de seña para reservar un auto?	Reserva y seña	0,765
★ ¿Cuántos años de garantía tiene un 0km?	Garantía de fábrica (0km)	0,762
¿Puedo entregar mi auto usado como parte de pago?	Entrega de tu usado como parte de pago	0,749
★ ¿Qué consumo de combustible tiene la pick-up Cordillera?	Pick-up Cordillera 4x4 Diésel	0,724
★ ¿Cuánto necesito de anticipo para financiar un auto?	Planes de financiación propia (30% mínimo)	0,646
¿Qué documentación tengo que llevar para comprar un auto?	Documentación requerida	0,643
¿Cada cuánto hay que hacer el service?	Service y mantenimiento programado	0,614
¿Me dan auto de cortesía mientras hacen el service?	Turnos de service	0,575
¿Cuántos caballos de fuerza tiene la Travesía?	SUV Travesía 2.0	0,505 (justo sobre el umbral)

B. Cruza dos documentos

Pregunta	Qué se espera	Similitud
★ Compré un usado certificado y lo quiero entregar como parte de pago de un 0km: ¿qué garantía tiene el usado certificado y hasta qué antigüedad me lo reciben?	Combina `garantia-service.md` (usados: 6 meses o 10.000 km) y `financiacion.md` (no se reciben usados de más de 15 años), citando las dos fuentes	0,754

C. No están: el sistema debe decir que no lo tiene

Pregunta	Qué pasa	Similitud
★ ¿Quién ganó el último mundial?	Cortada por el umbral, **sin llamar al modelo**	0,247
¿Cómo se hace una milanesa?	Cortada por el umbral, sin llamar al modelo	0,374

Pregunta	Qué pasa	Similitud
★ ¿Tienen autos eléctricos en el catálogo?	Cortada por el umbral (queda apenas abajo)	0,489

D. Casos límite: pasan el umbral aunque el dato no está

La búsqueda trae el tema general, pero no el dato puntual. Lo frena el **prompt**: el modelo dice que no lo encuentra o responde solo lo que el documento sí dice, sin inventar.

Pregunta	Qué se espera	Similitud
★ ¿Cuánto cuesta el service de la Vela 1.6?	Pasa el umbral. El documento dice que el costo "depende del modelo y se informa al momento del turno". Puede contestar "No lo encuentro" o contar solo lo que sí dice (primer service gratis), **sin inventar un precio**	0,693
¿Hacen envíos a domicilio del auto comprado?	Pasa el umbral. No hay dato de envíos: debería decir que no lo encuentra	0,586

Cómo usarlas en la exposición

1. **Una que funciona:** anticipo para financiar (abrir "Ver los documentos consultados" y mostrar la fuente).
2. **Una que no está:** mundial (mostrar la similitud contra la raya del umbral).
3. **Una que falla o queda en el límite:** service de la Vela 1.6 (explicar que la búsqueda trajo el tema, y que el prompt evitó el precio inventado).
4. **Extra:** la de los dos documentos.

El modelo no es 100% determinista: la redacción cambia entre corridas, pero las fuentes y las similitudes son siempre las mismas.